Smart Pantry Manager

1. Introduction

Households throw away food mainly because they forget what they have, or cannot think of something to cook with it. Smart Pantry Manager is an Android app, written in Java, that addresses this. The user records the ingredients in their pantry, and the app suggests recipes they can cook right now using only those ingredients. It then goes further: it warns about food that is about to expire, helps plan a week of meals around that food, and shows a dashboard of how much food and money has been used well or wasted.

The app works fully offline and stores its data in a local SQLite database.

1.1 Aim and objectives

- Let a user manage a pantry (add, view, edit and delete ingredients) with persistent storage.
- Suggest only recipes that can be cooked with the current pantry (strict matching).
- Reduce waste by highlighting food that is close to expiring and using it first.
- Connect the pantry and the recipes into one workflow: cook, plan and review.
- Show meaningful data about waste and savings.

1.2 Scope

The app is a single-user, offline, local-data application. It deliberately does not use maps, GPS or location features. It targets Android 6.0 (API 23) and above.

2. Requirements and features

2.1 Core requirements

- Pantry CRUD: name, quantity, unit and an optional expiry date.
- A library of recipes, each with a list of required ingredients and preparation steps. The app ships with 75 recipes.
- Strict-matching suggestions: a recipe is shown only when every ingredient is in the pantry in a sufficient quantity.
- Persistent storage using SQLite.
- Input validation and a bottom-navigation interface.

2.2 Additional features built

- **Expiry highlighting.** Expired items show in bold red. Items expiring within the warning window show in bold amber. The user chooses a window of 1, 3 or 7 days in Settings.
- **Use these first.** A card at the top of the pantry lists expired and soon-to-expire items, most urgent first.
- **Almost There.** Recipes missing exactly one ingredient are listed separately.
- **I cooked this.** Takes the used amounts out of the pantry and records them.
- **Recipe photos.** Every recipe has a photo, with a placeholder icon if an image is missing.
- **Weekly meal planner.** Breakfast, lunch and dinner for seven days, with recipes that use expiring ingredients offered first.
- **Waste and savings dashboard.** Items used before expiry, estimated money saved, money wasted, grocery spending, monthly charts and a list of expired or discarded items.
- **Price tracking.** An optional price when adding an item, which feeds the dashboard.

3. System design

3.1 Screens

- **Pantry:** list of ingredients, the Use these first card and an add button.
- **Add / Edit ingredient:** name, quantity, unit, price and expiry date, with validation.
- **Suggestions:** recipes that can be cooked now, and the Almost There list.
- **Recipe detail:** photo, ingredients, steps, a Back button and the I cooked this button.
- **Planner:** seven day cards, each with three meal slots, and a recipe picker.
- **Settings:** expiry alerts toggle, warning window, units preference and a link to the dashboard.
- **Dashboard:** statistics and charts.

The four main screens (Pantry, Suggestions, Planner, Settings) are reached through a bottom navigation bar. The other screens open from them.

[Insert screenshot of each screen here, with a one-line caption.]

3.2 Architecture and package structure

The code is organised by responsibility so that logic can be tested without a device.

- `activities`: one class per screen. They handle user input and display only.
- `adapters`: RecyclerView adapters for the pantry and recipe lists.

- `database`: `DatabaseHelper`, which holds all SQL, and `RecipeSeeder`, which inserts the 75 recipes.
- `models`: plain data classes (`PantryItem`, `Recipe`, `RecipeIngredient`, `MealPlanEntry`, `FoodEvent`).
- `utils`: the application logic: `IngredientMatcher`, `ExpiryUtils`, `PantryCooker`, `MealPlanner` and `WasteStats`. These classes avoid Android dependencies wherever possible.
- `views`: `BarChartView`, a custom chart drawn on a Canvas, so no third-party chart library is needed.

Keeping the decisions (what to suggest, what is expiring, what a plan needs) in the `utils` package means they can be verified with fast local unit tests, and the activities stay thin.

3.3 Technology choices

- **Java and AndroidX** with Material Components for the interface.
- **SQLite through SQLiteOpenHelper** for storage.
- **No third-party libraries** beyond AndroidX and Material.

4. Database design

4.1 Why SQLite

SQLite was chosen over a cloud database such as Firebase or PostgreSQL for three reasons. It needs no account, server or internet connection. Persistence is easy to demonstrate, because the data is still there after closing and reopening the app. And the data is naturally relational: a recipe has many ingredients, and a planner slot refers to a recipe.

4.2 Tables

pantry_items (what the user owns)

- `id` INTEGER, primary key, auto-increment
- `name` TEXT: normalised name used for matching (lower case, simple plurals removed)
- `display_name` TEXT: the name as the user typed it
- `quantity` REAL
- `unit` TEXT
- `expiry_date` TEXT in the form yyyy-MM-dd, optional
- `price` REAL, optional: total price paid for the quantity held

- `expiry_logged` INTEGER, default 0: set once an expired event has been recorded, so it is never counted twice

recipes

- `id` INTEGER, primary key
- `name` TEXT
- `steps` TEXT

recipe_ingredients

- `id` INTEGER, primary key
- `recipe_id` INTEGER, foreign key to `recipes.id`
- `ingredient_name` TEXT
- `required_quantity` REAL
- `unit` TEXT

meal_plan

- `id` INTEGER, primary key
- `day_index` INTEGER: 0 for Monday up to 6 for Sunday
- `meal_type` INTEGER: 0 breakfast, 1 lunch, 2 dinner
- `recipe_id` INTEGER
- A unique constraint on (`day_index` , `meal_type`) means a slot holds one recipe, and assigning a new recipe replaces the old one.

food_events (the log behind the dashboard)

- `id` INTEGER, primary key
- `event_type` INTEGER: purchased, used, expired or discarded
- `item_name` TEXT
- `quantity` REAL and `unit` TEXT
- `value` REAL: the money value of the event, 0 if no price was entered
- `event_date` TEXT in the form yyyy-MM-dd
- `before_expiry` INTEGER: for used events, whether the item was still in date

4.3 Relationships

- One recipe has many recipe ingredients (one to many).
- One recipe can appear in many meal plan slots (one to many).
- Pantry items are matched to recipe ingredients by normalised name rather than by a foreign key, because a pantry item is typed freely by the user.

[Insert the ER diagram here.]

4.4 Schema versions and migration

The database has gone through three versions. Version 1 held the pantry and recipes. Version 2 added `meal_plan`. Version 3 added the price and expiry-logged columns and the `food_events` table.

The original upgrade method deleted every table and recreated them, which would have wiped a user's pantry on each update. It was changed so an upgrade only adds what is new: older databases gain the new tables and columns and keep their data.

4.5 Keeping data consistent

Actions that change several tables use a database transaction. Cooking a recipe updates or deletes the pantry rows and writes the food-log events in one transaction, so the pantry and the log cannot disagree if something fails part-way.

5. Core logic

5.1 Strict matching

A recipe is suggested only if every ingredient is present in the pantry in at least the required quantity. Ingredient names are normalised (case, spacing and simple plurals) so that "Eggs" in the pantry matches "egg" in a recipe. Recipes missing exactly one ingredient go to the Almost There list.

Quantities are compared after unit conversion. Weights convert to grams (g, kg) and volumes to millilitres (ml, l, tsp at 5 ml, tbsp at 15 ml), so 1 kg of flour satisfies a recipe needing 500 g. Units that cannot be converted, such as pieces, are compared as plain numbers.

5.2 Expiry calculation

`ExpiryUtils` works out the number of days until an item expires. It converts both dates to a day number using only the year, month and day, so a day that is 23 or 25 hours long because of daylight saving cannot change the answer. Invalid dates such as 29 February in a non-leap year are rejected rather than guessed.

An item is expired when the count is negative, and expiring soon when it falls within the user's chosen window of 1, 3 or 7 days.

5.3 Cooking a recipe

`PantryCooker` first checks that the recipe can be cooked, using the same rules as the matcher, so anything the app suggests can always be cooked. It then converts each recipe amount into the unit the pantry item uses and works out what is left. Items that reach zero are removed. When part of an item is used, its stored price is reduced in proportion, so the value of what remains stays correct.

5.4 Weekly planner

`MealPlanner` ranks recipes for the picker. Recipes using the most soon-to-expire pantry items come first, then recipes with the fewest missing ingredients, then alphabetical order. Expired items are ignored, so the planner never encourages cooking with food that has gone off.

The same class contains the shopping-list calculation: it adds up what all planned meals need (a recipe planned twice counts twice), subtracts the pantry stock once, and lists only the shortfall. The shopping-list screen has not been built yet.

5.5 Waste and savings

`WasteStats` builds food-log events and totals them by month. The definitions used are:

- **Saved:** the value of food used while it was still in date.
- **Wasted:** the value of food that expired or was thrown away.
- **Spent:** the prices entered when items were added.

When an item passes its expiry date, an expired event is recorded once and the item is flagged, so deleting it later does not count it a second time. When the user removes an item they choose whether they used it up, threw it away, or added it by mistake, and only the first two are recorded.

Because the figures depend on prices entered by the user, they are estimates, and the dashboard says so.

6. Testing

6.1 Unit tests

The application logic is covered by 67 JUnit tests in four test classes. They run on the development machine without an emulator.

- **Expiry (ExpiryUtilsTest):** day differences, month and year boundaries, leap years including 2000 and 2100, invalid dates, daylight-saving changes in New York, Sydney and Lord Howe, and identical results in five different time zones.

- **Cooking (PantryCookerTest):** unit conversion, floating-point leftovers, and a simulated cook of every one of the 75 recipes (with exact amounts, double amounts, kilogram and litre pantry units, and each ingredient missing or short).
- **Planner (MealPlannerTest):** expiry-based ranking, tie-breaking, the shopping list for partial stock and repeated recipes, and a test that buying exactly the shopping list for a 21-meal week covers the plan.
- **Dashboard (WasteStatsTest):** event values, the expired item being counted once, monthly totals across a year boundary, and the waste rate.

6.2 A defect found by testing

The cooking tests found that the matcher had no conversion for teaspoons and tablespoons. A pantry holding 1 tablespoon of oil failed to satisfy a recipe needing 3 teaspoons, and cooking would have subtracted the wrong amount. Adding the conversions (1 tsp = 5 ml, 1 tbsp = 15 ml) fixed it.

6.3 Manual testing

[Complete this table with your own manual test results.]

- Add an ingredient with valid and invalid input (empty name, zero quantity, negative price): [result]
- Edit and delete an ingredient: [result]
- Close and reopen the app, and confirm data persists: [result]
- Add items so one recipe becomes cookable, and confirm it appears in Suggestions: [result]
- Remove one ingredient and confirm the recipe moves to Almost There: [result]
- Cook a recipe and confirm the pantry quantities change: [result]
- Add an item expiring tomorrow and confirm the amber highlight and the Use these first card: [result]
- Change the warning window in Settings and confirm the highlighting changes: [result]
- Assign recipes in the planner, leave the screen and return: [result]
- Open the dashboard after cooking and discarding items: [result]

7. Version control

The project is kept in a Git repository on GitHub: [link].

[Paste a screenshot of the commit history here, and list the main commits with a one-line description each. Commit after each feature so the history shows the project growing step by step.]

8. Challenges and reflection

- **Daylight saving and dates.** Subtracting two timestamps gives the wrong number of days on a 23-hour or 25-hour day. Using calendar fields only avoids the problem, and tests in several time zones confirm it.
- **Unit handling.** Matching 1 kg against 500 g, and tablespoons against teaspoons, needed a common base unit. The same rules had to be used in matching, cooking and the shopping list so the three never disagree.
- **Protecting user data.** Changing the database schema must not delete a user's pantry, which meant replacing the original upgrade logic with a non-destructive migration.
- **Counting waste once.** An expired item stays in the pantry until the user removes it, so a flag on the item was needed to avoid logging it as waste twice.
- **Large images.** A single oversized photograph could crash the screen that displays it, so all images were resized before being added.
- **Navigation highlighting.** The bottom bar showed the wrong selected tab after switching screens, because a tapped tab was marked selected on the screen being left. Returning false from the listener for navigation taps fixed it.

9. Limitations and future work

- Android notifications before food expires. Alerts currently appear inside the app only.
- A shopping-list screen. The calculation exists and is tested but has no screen.
- Applying the metric or imperial preference to the displayed quantities.
- Adding the user's own recipes, which would give full create, read, update and delete on a second table.
- Searching and sorting the pantry list.
- Combining duplicate pantry rows for the same ingredient.
- Cloud backup or sync between devices.

10. Conclusion

Smart Pantry Manager meets the core requirement of suggesting only recipes that can be cooked from what is in the pantry, and extends it into a connected workflow: warn about expiring food, cook, plan the week and review the results on a dashboard. Most of the logic sits in small classes that are verified by 67 automated tests, including a simulated cook of every recipe in the app. The main remaining work is notifications and the shopping-list screen.

References and credits

- Android Developers documentation: SQLiteOpenHelper, RecyclerView, Material Components. [Add the exact pages you used.]
- Recipe photographs: [source and licence].
- [Any other sources.]

Appendix A: How to run the project

1. Clone the repository and open it in Android Studio.
2. Wait for Gradle to finish syncing.
3. Select an emulator or a connected phone (API 23 or higher) and click Run.
4. The 75 recipes are added automatically the first time the app starts.
5. To run the unit tests, right-click the `test` folder and choose Run Tests, or run `./gradlew testDebugUnitTest`.

Appendix B: Screenshots

[Add full-size screenshots: pantry with expiry colours, add item, suggestions, Almost There, recipe detail, planner, recipe picker, dashboard with both charts, settings.]